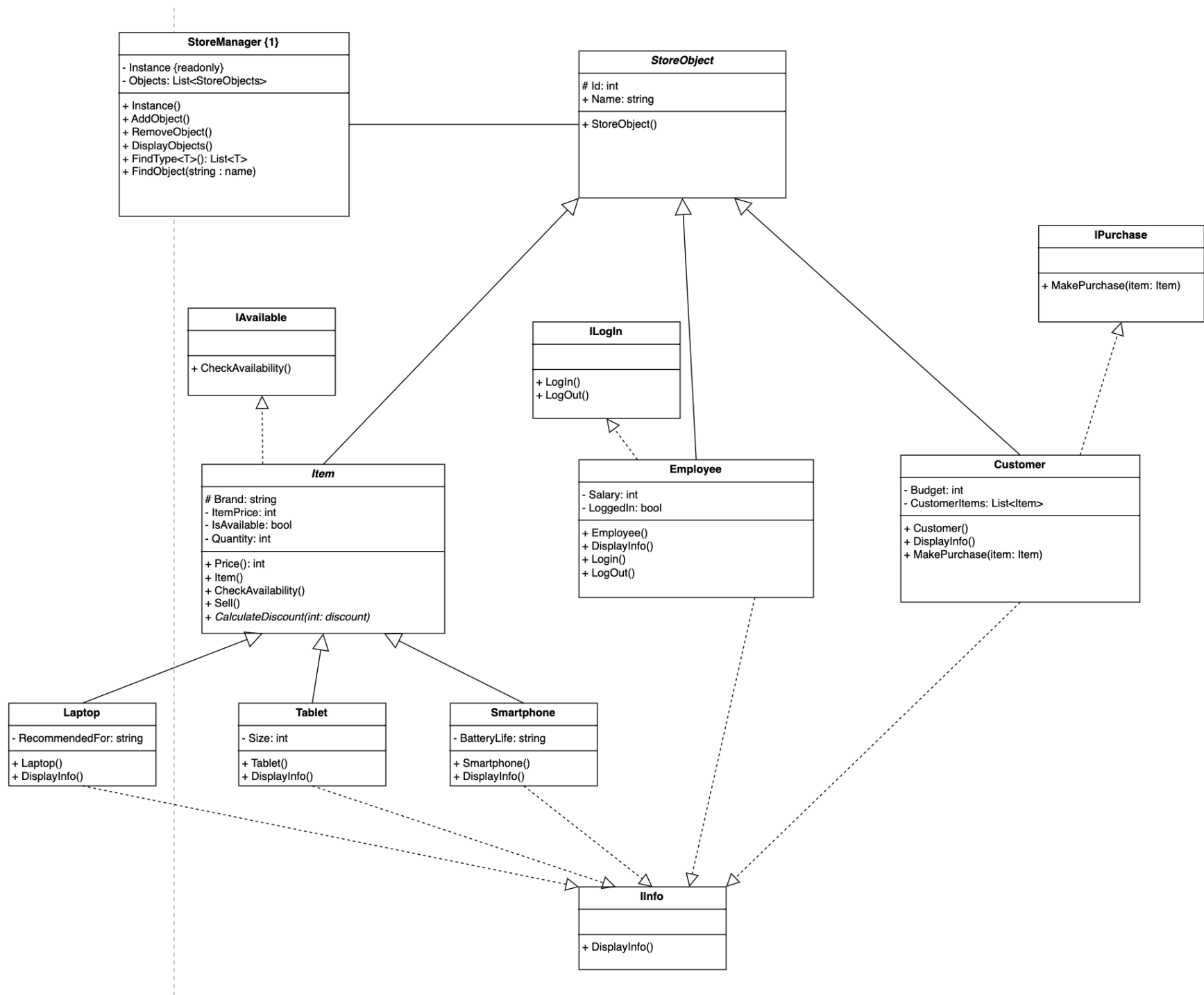


Introduction

The objective of the project is to design a store management system using object-oriented principles to handle items, employees, and customers. This report will cover the UML design choices, code structure, and adherence to object-oriented principles such as encapsulation, inheritance, polymorphism, and the use of interfaces.

System Overview

- Store Manager: Responsible for managing the objects in the store, including items, employees, and customers.
- Store Object: Responsible for representing all of the object classes in the store (employee, customer and item/product) in a universal manner
- Employee and Customer Management: Employees can log in and out, while customers can make purchases based on their budget.
- Product Inventory: Items in the store, such as laptops, tablets, and smartphones, can be added, viewed, and purchased by customers



UML Design Explanation

- Class Structure:
 - StoreManager: Designed as a singleton class to manage all store objects (employees, customers and items, which are split into laptop, tablet, and smartphone). Includes methods to add, remove, find, and display objects in the store. Maintains a register of objects, including items (goods), employees, and returning customers (assigning them Id in the store database)
 - StoreObject: The abstract base class for all entities in the system (items, employees, customers) with defining common properties: Id and Name.
 - Item: An abstract class inheriting from StoreObject to represent all store items, with attributes: Brand, ItemPrice, IsAvailable, and Quantity.
 - Laptop, Tablet, Smartphone: Specific types of items, each implementing additional attributes and inheriting from Item; they also implement inherited abstract method CalculateDiscount() from the Item class
 - Employee: Class representing employee entity, with specific attributes like Salary and Login, inheriting from StoreObject
 - Customer: Class representing returning customer entity, with specific attributes like Budget and CustomerItems, inheriting from StoreObject
- Interfaces:
 - IAvailable: Provides the CheckAvailability method to ensure an item can be purchased if it's in stock.
 - IInfo: Contains DisplayInfo, used by multiple classes to display relevant information; it allows specific classes (Laptop, Tablet, Smartphone, Employee, and Customer) to implement their own versions of DisplayInfo() without enforcing this requirement on the Item class directly with the abstract method implemented in StoreObject class.
 - ILogin: Defines methods for employees to allow them to log in and log out
 - IPurchase: Provides MakePurchase method for customers, allowing them to make purchases based on their budget

- Relationships:
 - Inheritance: inheritance is used to create a hierarchy with StoreObject as the base class for Item, Employee, and Customer. Subclasses inherit properties and methods while adding their own unique functionalities.
 - Interfaces: interfaces allow for flexible and modular functionality. For example, IAvailable allows any item to be checked for availability, while IPurchase and ILogin provide specific behaviours for customers and employees
- Encapsulation:
 - Private and protected attributes in classes, such as Quantity in Item and Budget in Customers, ensure that sensitive data is hidden and accessible only through methods and public properties, making sure it does not mistakenly change

Code Structure

- Singleton Pattern: The singleton design pattern was used for StoreManager to ensure there is only one instance managing all objects, reducing complexity and promoting global access to store data.
- Encapsulation: Each class uses an access modifier to restrict the visibility of sensitive data. For instance, ItemPrice in Item is private, with a public Price property to control access and make sure the value is not changed. Similarly, the Budget in Customer is private and modified only with MakePurchase.
- Constructors: Constructors are used in each class to initialise objects with required attributes.
- Polymorphism and Method Overriding:
 - Polymorphism is implemented through interfaces and base classes. Each item type (e.g., Laptop, Tablet, Smartphone) implements DisplayInfo from IInfo in a unique way to output specific details.
 - The FindType<T> method in StoreManager demonstrates polymorphism by allowing different subclasses of StoreObject to be retrieved dynamically. This method also uses Generics to avoid code redundancy.

- Interfaces and Modularity:
 - IAvailable, IInfo, ILogin, and IPurchase are used to promote modularity. By separating the functionality (e.g., CheckAvailability for items, MakePurchase for customers), the code remains organized, flexible, and easily extensible for future requirements.

Examples and Testing

- The system is initialised and used, as seen in the testing/logic code:
 - Adding Employees and Customers: Instances of Employee and Customer are created and managed by StoreManager.
 - Displaying Objects: The DisplayObjects method outputs the type and name of each object, showing how the system handles different types in a generic manner.
 - Finding and Filtering Items: FindType<Item> and FindObject are tested to demonstrate polymorphic behavior.
 - Purchasing Items: The MakePurchase method in Customer is tested with different scenarios (e.g., sufficient and insufficient budget), confirming that purchase logic, including budget checks and item availability, works as expected.
 - Applying discounts: Different discounts were applied to different items to demonstrate the use of the abstract method inherited by each Item

Design Choices and Justification

- Singleton Pattern for StoreManager:
 - The Singleton pattern was chosen for StoreManager to ensure a single, globally accessible instance for managing the store's data, avoiding potential conflicts with multiple instances.
- Use of Abstract Classes and Interfaces:
 - StoreObject is an abstract class for shared attributes like Id and Name, which simplifies inheritance.
 - Interfaces like IAvailable, IInfo, and ILogin allow for flexibility by enabling different classes to implement common methods without rigid inheritance.

- Inheritance Structure:
 - Item subclasses represent specific types of products and have unique attributes and methods. This structure enables code reuse while preserving flexibility for future expansion (e.g., adding new product types).

Object-Oriented Principles

- Encapsulation: Data encapsulation protects sensitive attributes, restricting access and modifications only through defined methods.
- Inheritance: Inheritance reduces redundancy, allowing shared properties and methods to be reused across subclasses.
- Polymorphism: Polymorphism is achieved with interfaces and overridden methods (e.g., `DisplayInfo`), enabling flexible behaviour for different item types.
- Abstraction: By using abstract classes and interfaces, the design hides implementation details, presenting only necessary functionality to the user.

Future Improvements

- Potential Improvements:
 - Adding features like automated inventory restocking, customer accounts with purchase history or an admin role for employees.
 - Adding exception handling for cases such as attempting to purchase an unavailable item.